Machine Learning

What machine learning is — Lecture 17 (Chapter 9, Part 1)

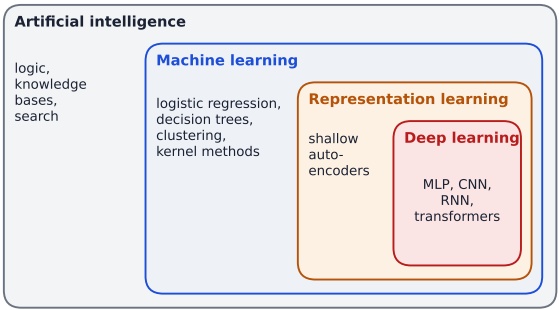
Amir Farbin

Part 1 — What machine learning is

From a test statistic to machine learning

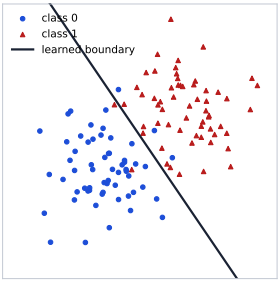
- A **test statistic** is one number computed from an event that you cut on. The best is the **likelihood ratio**: how much more probable the event is under signal than under background
- In one dimension you can histogram it. In 18 you can't: 10 bins per feature is 10^{18} bins, almost all of them empty: the **curse of dimensionality**
- The way out: **project** the 18 numbers down to **one**, and cut on that
  - by hand: **feature engineering**, designing new **features** (numbers that describe an event) like M_{TR}^2
  - learned from the data: **machine learning**

AI, machine learning, deep learning

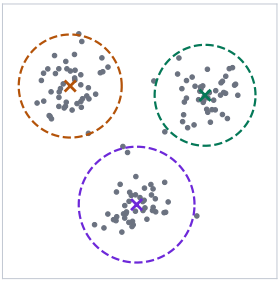


- **Artificial intelligence (AI):** any technique that mimics intelligent behavior. **Machine learning (ML):** a generic algorithm, made specific by **data**
- **Representation learning:** learning the features too. **Deep learning:** representation learning with many-layered neural networks

Supervised and unsupervised



supervised: learn y from x



unsupervised: find structure

- **Supervised:** each training example comes with its answer, its **label**. **Classification:** predict a category (cat or dog). **Regression:** predict a number (the dog's weight)
- **Unsupervised:** no labels. **Clustering:** group similar examples (e.g. **k-means**). **Anomaly detection:** flag what isn't normal. **PCA** (principal component analysis): find the directions the data varies most

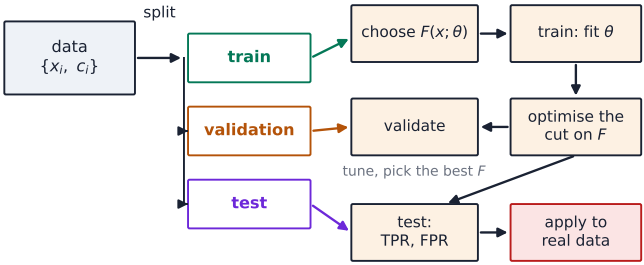
Do the simplest thing first

- **Look at the data**, then try **one cut**, then a **linear model**, then something fancy
- A **baseline**, the simple method's result, tells you whether a fancy method failed or the problem is just hard
- Pick the fancy one when you can make a case for it

The ingredients

- **Data:** examples of inputs, and for supervised learning the outputs you want
- **A model** $F(X \mid \theta)$: a function with dials, the **parameters** θ
- **A cost** (or **loss**) **function:** one number that says how bad F is on the data
  - supervised: compare F's answer with the truth
  - unsupervised: score similarity, or reproduce the input through a bottleneck ($F(X) = X$, an **autoencoder**)
- **An optimizer:** turns the dials until the cost is small. Doing that is **training**
- What training produces is **the parameters**. **Inference:** using the trained model on new data

The workflow for a signal search



The truth label c is used to train and to test F — never as an input. F sees only x.

Real data has no c: that is what F is for.

- F's output is **a new feature**, the model's **score**: histogram signal and background, pick the **cut** (threshold)

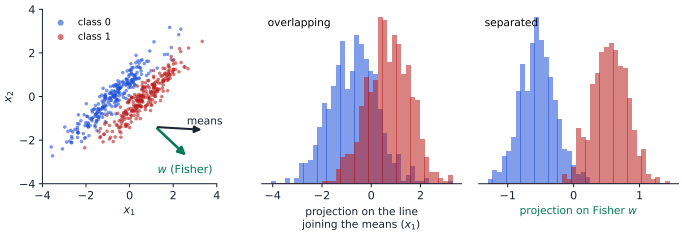
Train, validation, test

- **Training set:** fit θ
- **Validation set:** choose between models, settings (**hyperparameters**: choices you make, not learned) and the cut
- **Test set:** used **once**, at the end, with everything frozen
- Why three: a model can **memorize** its training data (**overfitting**), and if you tune on a sample you fit its **fluctuations**. What we want is **generalization**: good results on new data

Linear models

- $F(\vec{x}) = W\vec{x} + \vec{b}$: each output is a weighted sum of the inputs. W holds the **weights**, $\vec{b}$ the **biases**
- Find W and $\vec{b}$ by minimizing the **mean squared error** $\frac{1}{N}\sum_i^N \left(F(\vec{x}_i \mid W, \vec{b}) - \vec{y}_i\right)^2$
- Simple example: $\vec{b} = 0$, W is $1 \times d$. Stack the data: X has the N examples $\vec{x}_i$ as rows, Y the N answers
- **Analytic solution** (a formula, no iterating): $W = (X^T X)^{-1} X^T Y$. This is **least squares**

Fisher's linear discriminant (Lab 7)

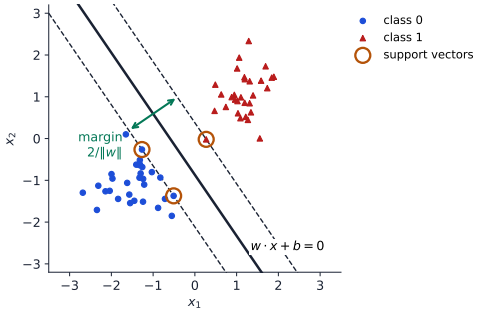


- Two classes with means $\vec{\mu}_0, \vec{\mu}_1$ and **covariance matrices** Σ_0, Σ_1 (how the features spread and vary together); model $y = W\vec{x}$: a **projection** onto one line
- **Maximize** $S = \dfrac{(W \cdot \vec{\mu}_0 - W \cdot \vec{\mu}_1)^2}{W^T\Sigma_0 W + W^T\Sigma_1 W}$: means far apart, each class narrow
- **Analytic solution:** $W = (\Sigma_0 + \Sigma_1)^{-1}(\vec{\mu}_0 - \vec{\mu}_1)$. Also called **linear discriminant analysis (LDA)**

Two losses, two analytic solutions

- **Mean squared error** $\rightarrow W = (X^T X)^{-1} X^T Y$: built from the **data** matrix X, $N \times d$
  - written in terms of the examples (the "dual" form, as the SVM uses next): $W = X^T (X X^T + \lambda I)^{-1} Y$, an inverse the **size of the data**, $N \times N$
- **Fisher's separation** $S \rightarrow (\Sigma_0 + \Sigma_1)^{-1}$: the **size of the features**, $d \times d$
  - 18×18 for SUSY, however many millions of events
- Almost nothing else in machine learning has a **closed form** (an analytic solution), so in general we train **numerically**: step by step

Every linear model draws a hyperplane

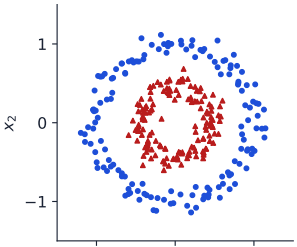


- A **hyperplane** is a flat boundary: a line in 2-D, a plane in 3-D. A **linear classifier** separates the classes with one
- **Support vector machine (SVM):** the **widest road** between the classes; the road's half-width is the **margin**
- Only the points on the edge of the road matter: the **support vectors**

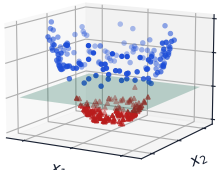
The SVM as template matching

- $f(x) = \sum_i a_i \, y_i \, (x_i \cdot x) + b$, with $y_i = \pm 1$ and most $a_i = 0$
- Compare the new event with the stored signal $(+)$ and background $(-)$ examples. The **dot product** $x_i \cdot x$ measures their similarity
- The **kernel** $k(x_i, x)$ replaces $x_i \cdot x$ by a similarity measured through a nonlinearity

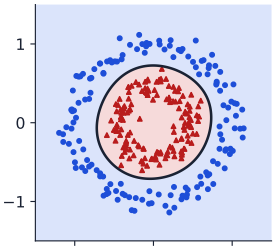
Kernels straighten a curved boundary



no straight line separates them



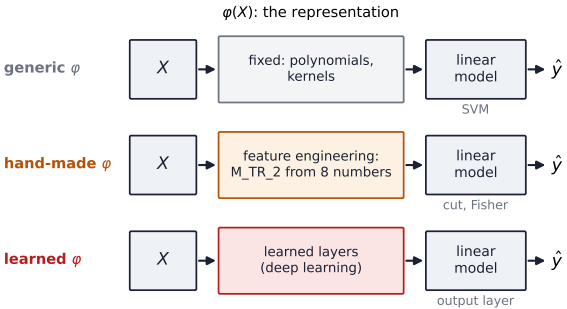
lifted: $\varphi(x) = (x_1, x_2, x_1^2 + x_2^2)$,
height $x_1^2 + x_2^2$: a plane separates them



RBF-kernel SVM boundary

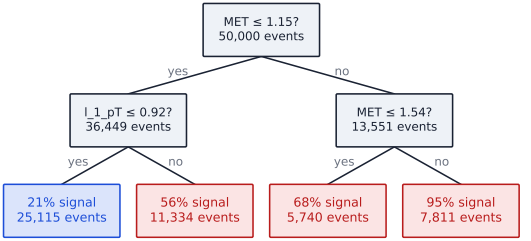
- Ring around a blob: no line in (x_1, x_2), one cut in $x_1^2 + x_2^2$
- ϕ maps the data into a new **feature space**. The **kernel trick**: ϕ is never computed, only $k(x_i, x)$. Polynomial and Gaussian (**RBF**, radial basis function) kernels

Features: generic, engineered, learned



- The detector's 10^8 channels $\rightarrow$ **reconstruction** (turning raw signals into particles) $\rightarrow$ **18 numbers**: SUSY is already engineered
- Every other method learns where to draw the boundary. **Deep learning learns the features**

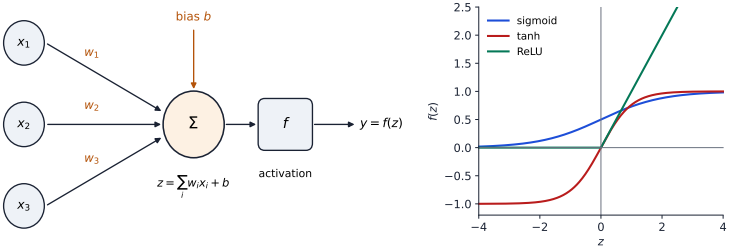
Decision trees



depth-2 tree offered M_TR_2, MET and l_1_pT; values in the dataset's own units

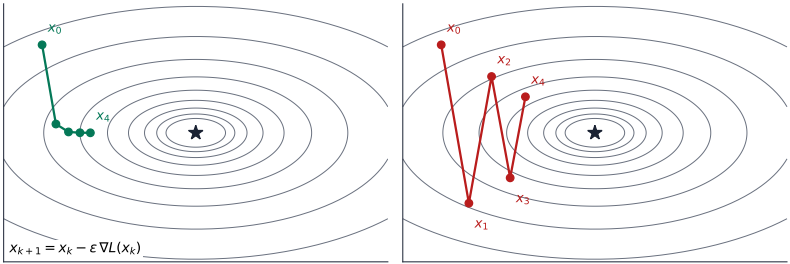
- **Decision tree:** cut on one feature at a time; each cut is a **node**, each final box a **leaf**. Split until the leaves are **pure** (one class)
- **Ensemble:** many models combined. **Random forest:** many trees **vote**. **Boosting:** each tree fixes the earlier ones' errors; **boosted decision trees (BDTs)**, e.g. XGBoost

A neuron



- A **neuron**: $F(x) = f(Wx + b)$, with f a **nonlinear activation function** (sigmoid, tanh, ReLU). If f were linear, we'd be back at the linear model
- **Universal approximation (1989): one hidden layer** (a layer between input and output) can approximate any continuous function, but nothing says how big it must be, or how to find it

Training is walking downhill



learning rate $\varepsilon = 0.09$: small: slow but steady

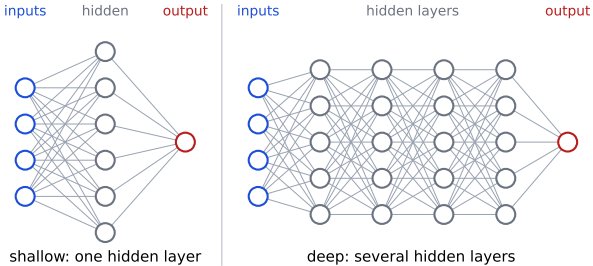
learning rate $\varepsilon = 0.18$: too large: zig-zags

- The cost $C(\theta)$ is a landscape over the parameters; start somewhere random. The **gradient** ∇C points uphill
- **Gradient descent:** $\theta \leftarrow \theta - \epsilon \nabla C$, with ϵ the **learning rate** (the step size)

Two views of learning

- **Frequentist:** there is a best θ; estimate it
- **Bayesian:** the data are fixed and θ is uncertain. Learn a **distribution** for each weight (a mean and a width) and get an **uncertainty** on the output
  - twice the parameters; **ensembles** of networks are the common alternative

Shallow and deep networks



- **Shallow:** one hidden layer. **Deep:** several. The weights are the **wiring**. For SUSY: 18 inputs, hidden layers, 1 output
- Depth: one layer **builds** nonlinear features, the next **combines** them

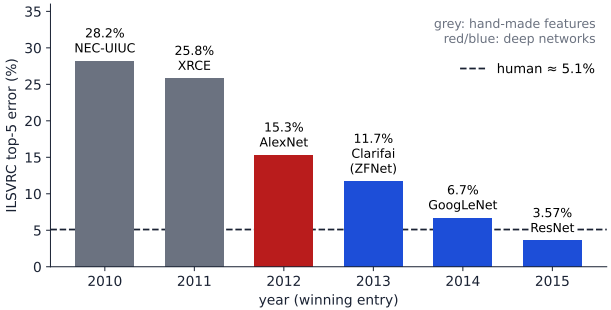
A short history

- **1943** McCulloch and Pitts: a mathematical neuron
- **1958** Rosenblatt's perceptron, a machine with motor-driven resistors for weights
- **1965** first trained multi-layer networks; **1986 backpropagation** (computing the gradient layer by layer, with the chain rule) popularized
- **1989** universal approximation; LeCun's network reads zip codes
- **1991** the **vanishing gradient**: in a deep network, the gradient for the early layers shrinks to almost nothing, so they barely learn
- 1990s–2000s: SVMs, boosted trees and symbolic AI take over

Why it works now

- **Activations: ReLU**, $\max(0, z)$: its slope is 1 wherever it's on. Plus better **initialization** (starting weights), layer-wise pre-training, normalization and **residual connections** (shortcuts past layers)
- **Data:** millions of labeled examples
- **GPUs** (graphics processing units): built for games, they do exactly the arithmetic training needs
- **2014:** deep learning reaches particle physics, in the SUSY dataset's own paper

2012 and after



- **ImageNet:** a million labeled photos in 1,000 categories. **Top-5 error:** how often the right answer isn't in the five best guesses
- Before 2012, vision ran on hand-made features. **AlexNet** (Toronto), a deep network, changed that
- Atari from pixels and the score (2013–15); AlphaGo beats Lee Sedol (2016)

Learning the features

- **Convolution:** slide a small **filter** (or **kernel**) over the image, multiplying and adding; it lights up where the pattern is
  - an identity-matrix filter is a **diagonal-line detector**
- Each filter gives a **feature map**; **pooling** shrinks it (keep the largest value in each small block); repeat: **local and general** $\rightarrow$ **global and specific**. This is a **convolutional neural network (CNN)**
- The filters are **learned**: edges, then parts, then objects
- **Transfer learning:** reuse a network trained on one task for another. **Fine-tuning:** start from that **pre-trained** network and retrain it a little, top layers first

Vocabulary (1)

- **test statistic, likelihood ratio, curse of dimensionality, feature, feature engineering**
- **artificial intelligence, machine learning, representation learning, deep learning**
- **supervised** (with **labels**) / **unsupervised; classification, regression, clustering, k-means, anomaly detection, PCA**
- **baseline, model, parameters, cost (loss) function, optimizer, training, inference, score, cut (threshold)**
- **training / validation / test set, hyperparameters, overfitting, generalization**
- **weights, biases, least squares, analytic (closed-form) solution, covariance matrix, projection, LDA**

Vocabulary (2)

- **hyperplane, linear classifier, support vector machine (SVM), margin, support vectors, dot product**
- **kernel, feature space, kernel trick, RBF**
- **reconstruction; decision tree, node, leaf, ensemble, random forest, boosting, BDT**
- **neuron, activation function, hidden layer, universal approximation, shallow / deep network**
- **gradient, gradient descent, learning rate, backpropagation, vanishing gradient, ReLU, initialization, residual connection, GPU**
- **frequentist / Bayesian; ImageNet, top-5 error; convolution, filter, feature map, pooling, CNN**
- **transfer learning, pre-trained, fine-tuning**